ASSIGNMENT 1 – CO1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs.

Student Name	G.RAJU	**Register Number**	192411065
Department / Section	CSE	**Academic Year / Semester**	
Faculty Name	Dr.Amirtha	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	09-11-2026	**Date of Evaluation**	
GitHub Repository	DSAO115-192411065	**Compiler**	g++ 13.3.0, -std=c++17 -Wall -Wextra

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		
9	Implementation and Testing	4		
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		

Task	Task Title	Max	Marks Awarded	Remarks
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	50		Percentage:

OVERALL REMARKS

Student Signature	**Faculty Signature**	**Program Director**

DSA01 – Object-Oriented Programming with C++

Assignment 1: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

1. Introduction

A City Metro Smart-Card System is an automated ticketing system that allows passengers to travel on a metro network without purchasing a separate paper ticket for every journey. Passengers recharge their smart cards with money and use them to enter and exit metro stations. The system calculates the fare according to the travel zone, time of travel, and applicable discounts.

The main purpose of this project is to design and implement a fare-calculation system using the principles of Object-Oriented Programming (OOP) in C++. The project demonstrates how real-world entities such as smart cards, passengers, journeys, and fare tables can be represented as classes and objects.

The assumed fares are ₹10 for the Inner Zone, ₹20 for the Middle Zone, and ₹30 for the Outer Zone. During peak hours, an additional surcharge of 20% is applied to the base fare. The system must also verify whether the passenger has sufficient balance before deducting the fare.

The system follows a minimum-balance rule that requires the card to retain at least ₹50 after a journey. If the balance after payment would be less than ₹50, the journey is rejected.

The project covers C++ concepts such as variables, constants, data types, operators, expressions, type conversion, conditional statements, functions, classes, objects, constructors, encapsulation, validation, and testing.

1.1 Objectives of the Project

1. To analyse the requirements of a metro smart-card fare system.
2. To identify the data required for fare calculation.
3. To apply appropriate C++ data types and constants.
4. To implement fare rules using arithmetic, relational, and logical operators.
5. To distinguish procedural programming from object-oriented programming.
6. To create classes that represent smart cards and journeys.

7. To protect card balance information through encapsulation.
8. To validate passenger input and prevent invalid transactions.
9. To test the system using normal, boundary, and invalid inputs.
10. To evaluate the correctness, efficiency, reliability, and maintainability of the program.

1.2 Problem Statement

The objective is to develop a C++ program that calculates the metro fare based on the selected travel zone and time of travel. The system accepts the required inputs, identifies the applicable fare, applies the peak-hour surcharge when necessary, checks the card balance, deducts the fare for a valid transaction, and displays the result.

The program should allow fare rules to be modified easily. For example, if the fare for a particular zone changes, the programmer should be able to update the fare table without rewriting the entire program.

The system must also prevent invalid inputs, negative balances, incorrect zone selections, and transactions that violate the minimum-balance rule.

1.3 Assumptions Used in the Design

Parameter	**Assumed rule**
Inner Zone base fare	₹10
Middle Zone base fare	₹20
Outer Zone base fare	₹30
Peak-hour surcharge	20%
Morning peak period	08:00–10:00
Evening peak period	17:00–20:00
Minimum balance after a journey	₹50

Initial card balance	₹100

These are assignment assumptions, not official fare rules for an actual metro network. Concession rates are not specified in the basic requirements.



Task 1: Problem Analysis and Object-Oriented Design

1.1 Analysis of the Problem

Problem analysis is the first stage of software development. It involves understanding what the system must do, identifying the required inputs and outputs, and determining the rules that control its operation.

For the metro fare system, the program accepts the selected zone, departure time, and current card balance. It calculates the correct fare and determines whether the passenger can complete the journey.

The problem is divided into the following tasks:

1. Accept the passenger's zone selection.
2. Validate the selected zone.
3. Accept the travel time.
4. Determine whether the travel time falls within a peak period.
5. Retrieve the base fare for the selected zone.
6. Calculate the applicable surcharge.
7. Determine the final fare.
8. Check the card balance.
9. Verify the minimum-balance requirement.
10. Deduct the fare if the transaction is valid.
11. Display the fare, remaining balance, and transaction status.

1.2 Identification of Inputs, Processing, and Outputs

Inputs

- Zone number selected by the passenger.
- Hour of travel.
- Current smart-card balance.
- Optional concession or pass information, if supported.

Processing

- Validate the selected zone.
- Identify the base fare.

- Check peak-hour conditions.
- Calculate the surcharge and final fare.
- Verify sufficient balance.
- Deduct the fare and update the card balance.

Outputs

- Selected zone.
- Base fare.
- Peak-hour status.
- Surcharge amount.
- Final fare.
- Remaining balance.
- Transaction success or failure message.

1.3 Identification of Classes and Objects

In OOP, a class is a blueprint used to create objects. An object is an instance of a class that contains data and provides operations on that data.

Class	Purpose	Example data or operations
SmartCard	Stores card balance and manages payments	balance, canPay(), deductFare()
Journey	Represents one metro journey	zone, hour, fare
FareTable	Stores base fares	getBaseFare()
FareCalculator	Calculates the final fare	calculateFare()
Passenger	Represents the card user	Passenger ID, card reference

For example, a SmartCard object may contain a balance of ₹500. A Journey object may represent travel to the Middle Zone at 09:00. The FareCalculator determines the fare, and the SmartCard checks whether the payment can be completed.

1.4 Procedural Programming and Object-Oriented Programming

Procedural programming organises a program mainly around functions and a sequence of operations. OOP organises a program around classes and objects that combine data and behaviour.

Procedural approach	Object-oriented approach
Uses functions as the main organisational units	Uses classes and objects
Data may be shared between many functions	Data can be protected inside objects
May use many global variables in large programs	Objects manage their own state
Changes to data structures may affect many functions	Changes can often be limited to a class
Suitable for small, straightforward programs	Useful for larger, expandable systems
Reuse mainly through functions	Reuse through classes, composition, and inheritance

For a small fare calculator, procedural programming may be sufficient. However, a complete smart-card system benefits from OOP because card balance, fare calculation, and journey details have different responsibilities.

1.5 Application of OOP Principles

Encapsulation

Encapsulation combines data and the operations that work on it within a class. It also controls access to internal data.

The card balance should be private. External code should not be allowed to change it directly to an arbitrary value.

class SmartCard {

```
private:
    int balancePaise;

public:
    SmartCard(int initialBalance)
        : balancePaise(initialBalance) {}

    int getBalance() const {
        return balancePaise;
    }
};
```

Here, balancePaise is private, while getBalance() provides controlled read access.

Abstraction

Abstraction exposes the operations a user needs while hiding unnecessary implementation details. A passenger needs to know the fare and remaining balance, not how the program internally identifies peak hours or performs calculations.

Inheritance

Inheritance allows a new class to reuse and extend an existing class. For example, a general Pass class could be extended by StudentPass or MonthlyPass. Inheritance should be used only when the relationship between classes is meaningful.

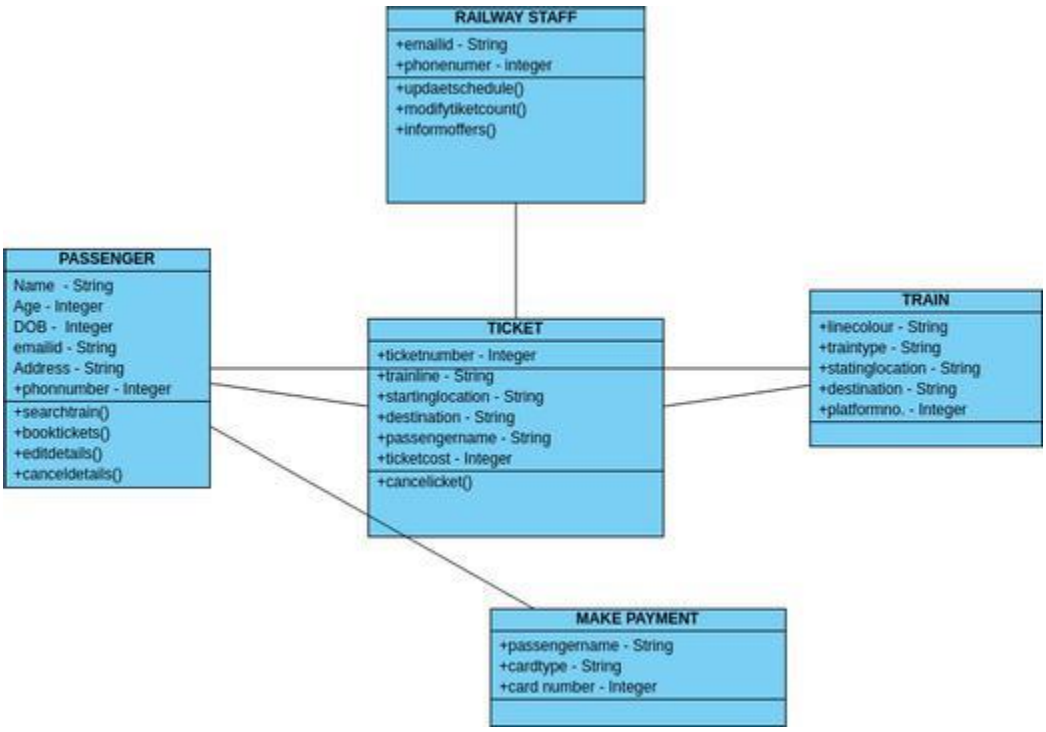
Polymorphism

Polymorphism allows the same interface to produce different behaviour depending on the object or implementation. Different pass types could implement different discount rules through a common interface.

1.6 Algorithm for the Metro Fare System

1. Start the program.
2. Read the zone number and travel hour.
3. Validate the zone number and time.
4. Select the base fare according to the zone.
5. Check whether the travel time falls within a peak period.
6. If it is peak time, calculate the 20% surcharge; otherwise, set the surcharge to zero.
7. Add the surcharge to the base fare.
8. Check whether the card can pay the fare while retaining at least ₹50.

9. If valid, deduct the fare and display the updated balance.
10. Otherwise, reject the transaction and display a message.
11. Stop the program.



Task 2: C++ Tokens, Constants, Enumerations, and Naming Conventions

2.1 Introduction to C++ Tokens

A token is the smallest meaningful unit of a C++ program recognised by the compiler. The main categories are keywords, identifiers, literals, operators, and punctuators.

1. Keywords

Keywords are reserved words with predefined meanings in C++. They cannot be used as ordinary variable or function names.

```
int balance;

if (balance > 50) {
    cout << "Sufficient balance";
}
```

Here, int and if are keywords. Other examples include class, private, public, return, const, while, for, and switch.

2. Identifiers

Identifiers are names given to variables, functions, classes, and other program elements.

```
int cardBalance;
double fare;

class SmartCard {};
```

Here, cardBalance, fare, and SmartCard are identifiers.

Rules for identifiers:

- They may contain letters, digits, and underscores.
- They cannot begin with a digit.
- They cannot contain spaces.
- They cannot be reserved keywords.
- They are case-sensitive.

For example, cardBalance, CardBalance, and cardbalance are different identifiers.

3. Literals

Literals are fixed values written directly in the source code.

```
int zone = 2;
double surcharge = 0.20;
char grade = 'A';
bool valid = true;
```

The values 2, 0.20, 'A', and true are literals.

4. Operators

Operators perform operations on values and variables.

- Arithmetic: +, -, *, /, %
- Relational: >, <, >=, <=, ==, !=
- Logical: &&, ||, !
- Assignment: =, +=, -=
- Increment and decrement: ++, --

Example:

```
int fare = 20;
fare = fare + 5;
```

The + operator adds the values, while = assigns the result.

5. Punctuators

Punctuators organise program syntax. Examples include semicolons, commas, braces, parentheses, and square brackets.

```
if (fare > 0) {
    cout << fare;
}
```

Parentheses define the condition, braces define the block, and the semicolon terminates the output statement.

2.2 Difference Between #define, const, and constexpr

Constants represent values that should not change during normal program execution.

Using #define

#define MIN_BALANCE 5000

The preprocessor replaces the symbolic name with the specified text before compilation. However, #define does not create a typed C++ variable and does not follow normal variable scope rules.

Using const

const int MIN_BALANCE_PAISE = 5000;

The const keyword declares a typed object that cannot be modified through that name after initialisation.

Using constexpr

constexpr int MIN_BALANCE_PAISE = 5000;
constexpr int INNER_FARE_PAISE = 1000;

constexpr indicates that a value can be used in constant expressions when the requirements are satisfied.

Feature	#define	const	constexpr
Mechanism	Preprocessor replacement	Typed C++ object	Compile-time constant when valid
Has a data type	No inherent type	Yes	Yes
Follows normal scope rules	Not like variables	Yes	Yes
Useful for compile-time values	Yes	Sometimes	Yes
Preferred for modern C++ constants	Usually not for ordinary values	Yes	Yes, when compile-time evaluation is intended

For this project, constexpr is suitable for fixed fares and minimum-balance rules.

2.3 Using enum class for Zones

An enumeration represents a group of named values. It improves readability by replacing unexplained numbers with meaningful names.

```
enum class Zone {
    Inner = 1,
    Middle = 2,
    Outer = 3
};
```

Instead of writing zone == 2, the program can use:

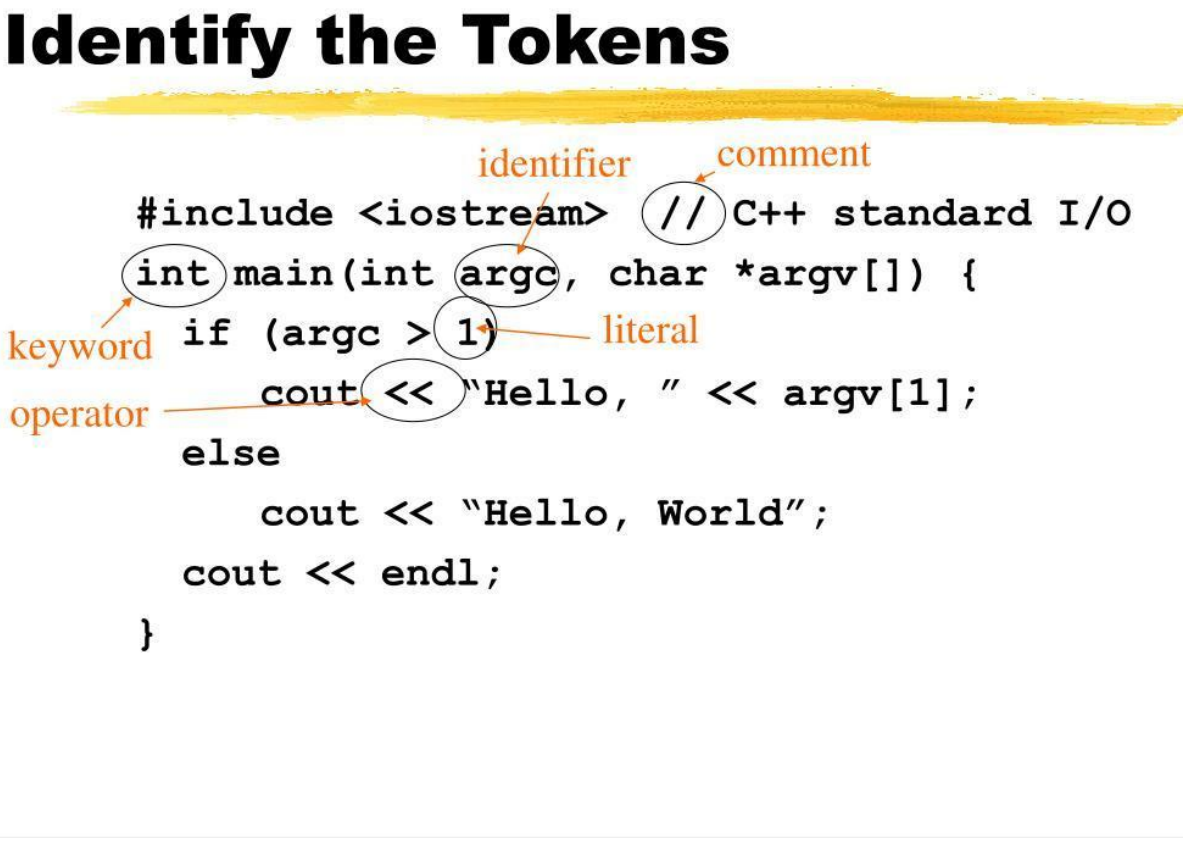
```
if (zone == Zone::Middle) {
    // Process Middle Zone fare
}
```

Zone::Middle communicates the meaning clearly. The enum class form is generally safer because its values are scoped and do not automatically convert to integers.

2.4 Naming Conventions for the Project

Program element	Recommended example
Class name	SmartCard
Function name	calculateFare()
Variable name	currentBalance
Constant name	MIN_BALANCE_PAISE
Enumeration type	Zone
Enumeration value	Zone::Inner

Meaningful names should be preferred over unclear names such as x, y, and z, except where short names are conventional in mathematical expressions.



Task 3: Data Types, Memory, and Type Conversion

3.1 Importance of Data Types

A data type determines the kind of value a variable can store, the operations that can be performed on it, and its representable range.

The metro fare system handles money, travel time, zone identifiers, and transaction status. Therefore, suitable data types must be selected for each value.

3.2 Suitable Data Types

Data type	Example	Use in the metro system
int	int hour = 9;	Hour, zone number, fare in paise
long long	long long balance = 1000000;	Large monetary values or totals
double	double rate = 0.20;	Percentage calculations
char	char option = 'Y';	Single-character menu option
bool	bool isPeak = true;	Peak-hour or validation status
string	string cardId = "C101";	Card identifier or passenger name
enum class	Zone::Outer	Named travel zones

The exact memory size of a type can depend on the implementation. A programmer should not assume that every system uses identical sizes for all types.

3.3 Why Money Should Be Stored as Integer Paise

Using floating-point numbers for money can introduce rounding differences because many decimal fractions cannot be represented exactly in binary floating-point format.

For example, ₹10.50 can be represented as 1050 paise. Storing money as integer paise avoids many floating-point representation errors.

```
int farePaise = 2000;      // ₹20.00
int balancePaise = 10000;  // ₹100.00

balancePaise -= farePaise;
```

After deduction, the balance is 8000 paise, equivalent to ₹80.00.

For a production system, the programmer must also define a rounding policy for percentage calculations.

3.4 Implicit Type Conversion

Implicit conversion occurs when the compiler automatically converts a value from one type to another.

```
#include <iostream>
using namespace std;

int main() {
   int fare = 20;
   double amount = fare;

   cout << amount;
   return 0;
}
```

Output:

```
20
```

The integer value 20 is converted to a double value.

However, converting a floating-point value to an integer may discard its fractional part.

```
double amount = 20.75;
int fare = amount;

cout << fare;
```

Output:

The fractional portion is discarded; it is not automatically rounded to the nearest integer.

3.5 Explicit Type Conversion

Explicit conversion occurs when the programmer requests a conversion.

```
double amount = 20.75;
int fare = static_cast<int>(amount);
```

The expression static_cast<int>(amount) converts the value to an integer. In this case, the result is 20.

Explicit conversion makes the programmer's intention clear, but the value must still be valid for the destination type.

3.6 Integer Overflow

Integer overflow occurs when an arithmetic result cannot be represented by the relevant integer type. Signed integer overflow in C++ is undefined behaviour, so a program should avoid it.

Example:

```
int total = 2000000000;
total = total + 1000000000;
```

The mathematical result is 3,000,000,000, which exceeds the maximum value of a typical 32-bit signed int.

Safer practices include:

- Selecting an appropriate integer type.
- Validating user input.
- Checking arithmetic bounds.

Task 4: Expressions and Type Conversion in C++

Expressions combine variables, literals, and operators to produce a value. The result depends on operator precedence, associativity, and operand types.

4.1 Arithmetic Expression

```
int baseFare = 2000;
int surcharge = 400;
int finalFare = baseFare + surcharge;
```

The base fare is 2000 paise and the surcharge is 400 paise. The final fare is 2400 paise, equivalent to ₹24.

4.2 Relational Expression

A relational expression compares two values and produces a Boolean result.

```
int balance = 10000;
bool sufficient = balance >= 5000;
```

Since 10000 is greater than 5000, sufficient becomes true.

Relational operators include:

- > greater than
- < less than
- >= greater than or equal to
- <= less than or equal to
- == equal to
- != not equal to

The difference between = and == is important. The first assigns a value, while the second compares two values.

4.3 Logical Expression for Peak Hours

The peak periods are 08:00–10:00 and 17:00–20:00. If the system stores the travel hour as an integer, it can check the following condition:

```
bool isPeak =
    (hour >= 8 && hour < 10) ||
    (hour >= 17 && hour < 20);
```

The && operator requires both conditions in a group to be true. The || operator requires at least one group to be true.

For example:

- When hour = 9, isPeak is true.
- When hour = 14, isPeak is false.

The intervals include the starting hour and exclude the ending hour. Therefore, 09:00 is peak time, while 10:00 is not included in the morning peak period.

4.4 Type Conversion in Expressions

Consider the following example:

```
int fare = 3;
int count = 2;
double average = fare / count;
```

Both operands in fare / count are integers. Integer division occurs first, producing 1, which is then converted to double. The value stored in average is 1.0, not 1.5.

To perform floating-point division, at least one operand must be converted before division:

```
double average = static_cast<double>(fare) / count;
```

The result is 1.5.

Although floating-point arithmetic is useful for many calculations, the fare system should preferably use integer paise for the final monetary amount.

Task 5: Operators, Precedence, and Fare Calculation

5.1 Arithmetic Operators

Arithmetic operators perform mathematical operations on numerical values.

Operator	Operation	Example	Result
+	Addition	20 + 5	25
-	Subtraction	20 - 5	15
*	Multiplication	20 * 2	40
/	Division	20 / 4	5
%	Remainder	20 % 3	2

In the metro fare system, addition adds the surcharge to the base fare, subtraction deducts the fare from the balance, and multiplication can calculate percentage-based amounts.

5.2 Operator Precedence

Operator precedence determines which operation is performed first.

int result = 10 + 20 * 2;

Multiplication has higher precedence than addition. Therefore:

$$10 + (20 \times 2) = 50$$

The result is 50, not 60.

Parentheses can make the intended order explicit:

int result = (10 + 20) * 2;

The result is 60.

5.3 Calculating the Peak-Hour Surcharge

The surcharge is 20% of the base fare.

$$\square\square\square\square h\square\square\square\square = \square\square\square\square\square\square\square\square\square \times \frac{20}{100}$$

For a Middle Zone journey with a base fare of ₹20:

$$\square\square\square\square h\square\square\square\square = 20 \times \frac{20}{100} = ₹4$$

Therefore:

$$\square\square\square\square\square\square\square\square\square\square = 20 + 4 = ₹24$$

If the same journey occurs outside peak hours, the surcharge is zero, and the final fare remains ₹20.

5.4 Increment and Decrement Operators

The increment operator ++ increases a variable by one. The decrement operator -- decreases it by one.

```
int journeys = 0;
journeys++;
```

After execution, journeys equals 1.

The prefix form ++journeys increments before the expression's value is used. The postfix form journeys++ uses the original value in the expression and increments afterward.

```
int count = 5;
int a = count++;
int b = ++count;
```

The resulting values are:

- a = 5
- count = 7
- b = 7

Increment operators may be used to count completed journeys, but complicated expressions that modify variables multiple times should be avoided.

5.5 Conditional Operator

The conditional operator ?: selects between two expressions.

int surchargePaise = isPeak ? baseFarePaise / 5 : 0;

This applies a 20% surcharge when isPeak is true and zero surcharge otherwise.

5.6 Fare Calculation Using Integer Arithmetic

Suppose the Middle Zone base fare is 2000 paise. The peak-hour surcharge is 20%.

$$\square\square\square\square h\square\square\square\square\square\square\square\square\square\square = \frac{2000 \times 20}{100} = 400$$

$$\square\square\square\square\square\square\square\square\square\square = 2000 + 400 = 2400\square\square\square\square\square$$

The final fare is ₹24. For larger applications, the programmer should validate values and use an appropriate integer type to avoid overflow.

Task 6: Algorithm, Pseudocode, and C++ Implementation

6.1 Algorithm

1. Start.
2. Declare constants for the fares and minimum balance.
3. Read the zone number and hour.
4. Validate the zone and hour.
5. Select the corresponding base fare.
6. Check whether the hour falls within a peak period.
7. Calculate the surcharge if the journey occurs during peak hours.
8. Calculate the final fare.
9. Check the current balance.
10. Verify that the balance after payment will be at least ₹50.
11. If the check succeeds, deduct the fare.
12. Display the base fare, surcharge, final fare, and balance.
13. Otherwise, display a rejection message.
14. Stop.

6.2 Pseudocode

```
BEGIN

   SET INNER_FARE = 1000 paise
   SET MIDDLE_FARE = 2000 paise
   SET OUTER_FARE = 3000 paise
   SET MIN_BALANCE = 5000 paise
   SET BALANCE = 10000 paise

   READ zone
   READ hour

   IF zone is invalid OR hour is invalid THEN
      DISPLAY "Invalid input"
      STOP
   END IF

   IF zone = 1 THEN
```

```
    SET baseFare = INNER_FARE
  ELSE IF zone = 2 THEN
    SET baseFare = MIDDLE_FARE
  ELSE
    SET baseFare = OUTER_FARE
  END IF

  SET isPeak = false

  IF hour is between 8 and 9
    OR hour is between 17 and 19 THEN
    SET isPeak = true
  END IF

  SET surcharge = 0

  IF isPeak = true THEN
    SET surcharge = baseFare * 20 / 100
  END IF

  SET finalFare = baseFare + surcharge

  IF BALANCE - finalFare >= MIN_BALANCE THEN
    SET BALANCE = BALANCE - finalFare
    DISPLAY "Journey approved"
    DISPLAY finalFare
    DISPLAY BALANCE
  ELSE
    DISPLAY "Insufficient balance"
  END IF

END
```

6.3 C++ Program

The following program uses integer paise, validates the zone and hour, calculates the surcharge, and applies the minimum-balance condition.

```
#include <iostream>
using namespace std;
```

```
constexpr int INNER_FARE = 1000;
constexpr int MIDDLE_FARE = 2000;
constexpr int OUTER_FARE = 3000;
constexpr int MIN_BALANCE = 5000;

int main() {
    int zone, hour;
    int balance = 10000;

    cout << "Enter zone (1-Inner, 2-Middle, 3-Outer): ";
    if (!(cin >> zone)) {
        cout << "Invalid zone input\n";
        return 1;
    }

    cout << "Enter travel hour (0-23): ";
    if (!(cin >> hour)) {
        cout << "Invalid hour input\n";
        return 1;
    }

    if (zone < 1 || zone > 3 || hour < 0 || hour > 23) {
        cout << "Invalid zone or hour\n";
        return 1;
    }

    int baseFare;

    if (zone == 1)
        baseFare = INNER_FARE;
    else if (zone == 2)
        baseFare = MIDDLE_FARE;
    else
        baseFare = OUTER_FARE;

    bool isPeak =
        (hour >= 8 && hour < 10) ||
        (hour >= 17 && hour < 20);

    int surcharge = isPeak ? baseFare / 5 : 0;
```

```
    int finalFare = baseFare + surcharge;

    cout << "\nBase fare: Rs " << baseFare / 100
         << "." << (baseFare % 100 < 10 ? "0" : "")
         << baseFare % 100 << '\n';

    cout << "Surcharge: Rs " << surcharge / 100
         << "." << (surcharge % 100 < 10 ? "0" : "")
         << surcharge % 100 << '\n';

    cout << "Final fare: Rs " << finalFare / 100
         << "." << (finalFare % 100 < 10 ? "0" : "")
         << finalFare % 100 << '\n';

    if (balance - finalFare >= MIN_BALANCE) {
        balance -= finalFare;

        cout << "Journey approved\n";
        cout << "Remaining balance: Rs "
             << balance / 100 << "."
             << (balance % 100 < 10 ? "0" : "")
             << balance % 100 << '\n';
    } else {
        cout << "Journey rejected: minimum balance rule violated\n";
    }

    return 0;
}
```

6.4 Sample Input

```
2
9
```

The passenger selects the Middle Zone and travels at 09:00. The initial balance is ₹100.

6.5 Sample Output

```
Enter zone (1-Inner, 2-Middle, 3-Outer): 2
Enter travel hour (0-23): 9

Base fare: Rs 20.00
```

Surcharge: Rs 4.00
Final fare: Rs 24.00
Journey approved
Remaining balance: Rs 76.00

6.6 Explanation of the Output

The selected zone is the Middle Zone, so the base fare is ₹20. The hour 09:00 lies within the morning peak period. Therefore, the surcharge is 20% of ₹20, which is ₹4.

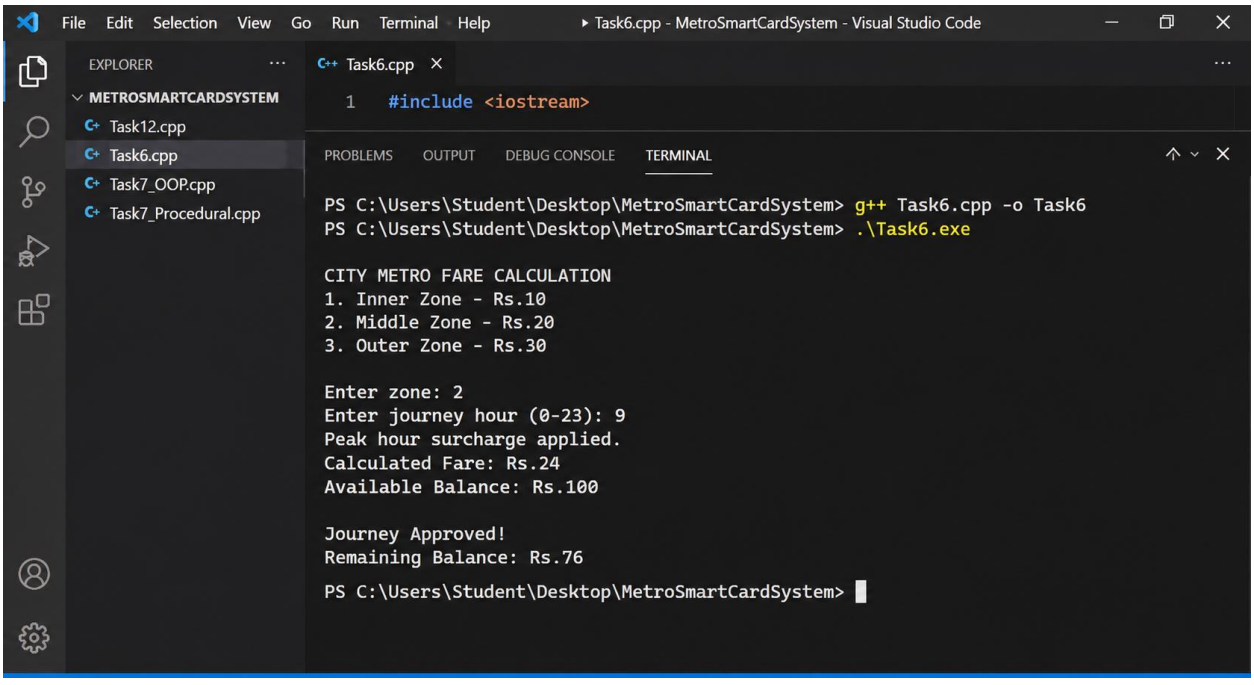
The total fare is:

$$₹20 + ₹4 = ₹24$$

The initial balance is ₹100, so the remaining balance is:

$$₹100 - ₹24 = ₹76$$

Since ₹76 is greater than the required minimum balance of ₹50, the journey is approved.



Task 7: Procedural and Object-Oriented Implementation

7.1 Procedural Programming Approach

In procedural programming, the program is divided into functions. Each function performs a specific operation.

The metro fare system may contain these functions:

- getBaseFare() returns the fare for a selected zone.
- isPeakHour() determines whether a journey occurs during peak hours.
- calculateFare() calculates the final fare.
- processPayment() checks the balance and deducts the fare.

Example Program

```
#include <iostream>
using namespace std;

int getBaseFare(int zone) {
    if (zone == 1)
        return 1000;
    if (zone == 2)
        return 2000;
    if (zone == 3)
        return 3000;

    return -1;
}

bool isPeakHour(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}

int calculateFare(int zone, int hour) {
    int baseFare = getBaseFare(zone);

    if (baseFare < 0)
```

```
        return -1;

    if (isPeakHour(hour))
        return baseFare + baseFare / 5;

    return baseFare;
}

int main() {
    int zone = 2;
    int hour = 9;

    int fare = calculateFare(zone, hour);

    if (fare >= 0)
        cout << "Fare in paise: " << fare;
    else
        cout << "Invalid zone";

    return 0;
}
```

Output:

Fare in paise: 2400

The program uses separate functions for the main operations. This structure is suitable for a small fare calculator. However, managing many cards, journeys, concession types, and transaction records may become difficult as the system grows.

7.2 Object-Oriented Programming Approach

In OOP, data and related operations are grouped into classes. A SmartCard class manages the balance, while a FareCalculator class calculates the fare.

Example Program

```
#include <iostream>
using namespace std;

class SmartCard {
private:
```

```
    int balancePaise;

public:
    SmartCard(int initialBalance)
        : balancePaise(initialBalance) {}

    int getBalance() const {
        return balancePaise;
    }

    bool canPay(int farePaise) const {
        return farePaise >= 0 &&
               balancePaise - farePaise >= 5000;
    }

    bool payFare(int farePaise) {
        if (!canPay(farePaise))
            return false;

        balancePaise -= farePaise;
        return true;
    }
};

class FareCalculator {
public:
    int calculateFare(int zone, int hour) const {
        int baseFare;

        if (zone == 1)
            baseFare = 1000;
        else if (zone == 2)
            baseFare = 2000;
        else if (zone == 3)
            baseFare = 3000;
        else
            return -1;

        bool peak =
            (hour >= 8 && hour < 10) ||
```

```
            (hour >= 17 && hour < 20);

        return baseFare + (peak ? baseFare / 5 : 0);
    }
};

int main() {
    SmartCard card(10000);
    FareCalculator calculator;

    int fare = calculator.calculateFare(2, 9);

    if (fare >= 0 && card.payFare(fare)) {
        cout << "Fare: Rs 24.00\n";
        cout << "Balance in paise: "
             << card.getBalance() << '\n';
    } else {
        cout << "Transaction rejected\n";
    }

    return 0;
}
```

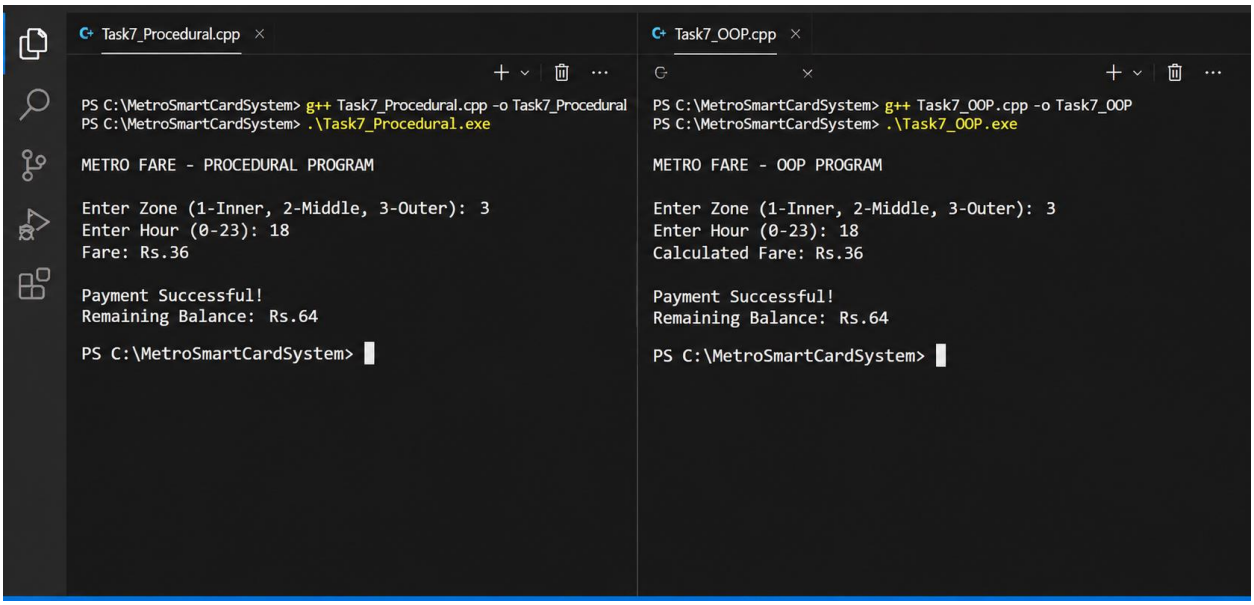
Output:

```
Fare: Rs 24.00
Balance in paise: 7600
```

7.3 Advantages of the OOP Approach

1. Data protection: The card balance is private.
2. Modularity: Fare calculation and card payment are handled by separate classes.
3. Reusability: Multiple smart-card objects can be created.
4. Maintainability: Fare rules can be changed without rewriting card-management logic.
5. Extensibility: Journey history and concession passes can be added.
6. Readability: Class and function names represent real-world responsibilities.

The procedural approach is simple, while the OOP approach provides better data organisation for a system that may expand over time.



Task 8: Input Validation, Error Handling, and Output Formatting

8.1 Importance of Input Validation

Input validation checks whether the values entered by a user satisfy the requirements of the program.

The fare system should not accept an invalid zone, a negative hour, or an hour greater than 23. Invalid inputs must be rejected before fare calculation.

For example, if a passenger enters zone number 5, the program should not automatically assign the Outer Zone fare. It should display an error because the selected zone is not defined.

8.2 Types of Validation

Range Validation

Range validation checks whether a value lies within the permitted range.

```
if (hour < 0 || hour > 23) {
    cout << "Invalid travel hour";
}
```

Membership Validation

Membership validation checks whether a value belongs to a valid set.

```
if (zone != 1 && zone != 2 && zone != 3) {
   cout << "Invalid zone";
}
```

Balance Validation

Balance validation checks whether the passenger can pay the fare while maintaining the minimum required balance.

```
if (balance - fare >= 5000) {
   balance -= fare;
} else {
   cout << "Insufficient balance";
}
```

A complete implementation should also check that the fare is non-negative and the balance is valid before subtraction.

8.3 Boundary Conditions

Boundary conditions are values at or near the limits of the permitted range.

The morning peak period is defined as 08:00–10:00, including 08:00 but excluding 10:00.

Travel hour	Expected status
7	Off-peak
8	Peak
9	Peak
10	Off-peak

16	Off-peak
17	Peak
19	Peak
20	Off-peak

This table verifies the behaviour at the beginning and end of each peak interval.

8.4 Output Formatting

Monetary values should be displayed consistently. A fare of 1000 paise should appear as ₹10.00, while 1025 paise should appear as ₹10.25.

```
int amount = 1025;

cout << "Rs " << amount / 100 << "."
     << (amount % 100 < 10 ? "0" : "")
     << amount % 100;
```

Output:

Rs 10.25

Integer division obtains the rupee component, while the remainder obtains the paise component.

8.5 Error Handling

The system should display meaningful messages for different failure conditions.

Error condition	Suggested message
Invalid zone	Invalid zone selection
Invalid hour	Travel hour must be between 0 and 23

Invalid numeric input	Please enter a valid number
Insufficient balance	Transaction rejected: insufficient balance
Minimum balance violation	Minimum balance requirement not satisfied

Task 9: Test Cases and Validation of Results

9.1 Purpose of Testing

Software testing executes a program with selected inputs to verify that it produces the expected outputs.

A program may compile successfully but still produce incorrect results. Therefore, the fare system must be tested for fare calculation, balance rules, peak-hour boundaries, and invalid inputs.

The main testing types are:

- Normal testing: Uses common, valid inputs.
- Boundary testing: Checks values near limits.
- Negative testing: Uses invalid inputs.
- Integration testing: Checks fare calculation and card payment together.
- Regression testing: Checks that existing behaviour remains correct after code changes.

9.2 Fare Calculation Test Cases

Assume the initial card balance is ₹100 and the minimum balance is ₹50.

Test	Zone	Hour	Base fare	Surcharge	Final fare	Expected result
1	Inner	12	₹10	₹0	₹10	Fare calculated
2	Middle	9	₹20	₹4	₹24	Fare calculated
3	Outer	18	₹30	₹6	₹36	Fare calculated
4	Inner	8	₹10	₹2	₹12	Peak surcharge applied
5	Middle	10	₹20	₹0	₹20	Morning peak ended
6	Outer	20	₹30	₹0	₹30	Evening peak ended
7	Invalid zone	9	—	—	—	Input rejected
8	Middle	24	—	—	—	Input rejected

9.3 Balance Validation Test Cases

The minimum-balance rule requires the balance after payment to be at least ₹50.

Initial balance	Fare	Balance after payment	Expected result
₹100	₹24	₹76	Accepted
₹80	₹30	₹50	Accepted
₹79	₹30	₹49	Rejected
₹50	₹10	₹40	Rejected

₹200	₹36	₹164	Accepted

A final balance of exactly ₹50 is valid because the rule requires the remaining balance to be greater than or equal to ₹50. A balance of ₹49 must be rejected.

9.4 Example of Multiple Journeys

Suppose a passenger starts with ₹200 and completes the following five journeys.

Journey	Zone	Hour	Fare
1	Inner	12	₹10
2	Middle	9	₹24
3	Outer	14	₹30
4	Inner	18	₹12
5	Middle	11	₹20
Total			₹96

The total fare is:

$$₹10 + ₹24 + ₹30 + ₹12 + ₹20 = ₹96$$

The final balance is:

$$₹200 - ₹96 = ₹104$$

Since the balance remains above ₹50 after every journey, all five transactions satisfy the minimum-balance rule.

9.5 Testing Procedure

1. Compile the program and check for compilation errors.
2. Execute it using a valid zone and hour.
3. Compare the displayed fare with the expected fare.
4. Process a transaction using a known starting balance.
5. Check whether the correct amount is deducted.
6. Test values at peak-hour boundaries.
7. Test a balance that leaves exactly ₹50.
8. Test a balance that leaves ₹49.
9. Enter invalid zone and hour values.
10. Record the actual output and compare it with the expected output.

Task 10: Compiler, Code Quality, and Performance

10.1 Compilation Process

A C++ program must be translated into executable machine code before it can run. The development process includes preprocessing, compilation, assembly, and linking.

1. Preprocessing: Processes directives such as #include.
2. Compilation: Checks syntax and translates the C++ source into lower-level code.
3. Assembly: Produces object code.
4. Linking: Combines object code and required libraries to create an executable program.

If a program contains a syntax error, compilation may fail. If it compiles but contains an incorrect fare formula, the error is logical rather than syntactical.

10.2 Compiler Command

A typical command for compiling the program is:

g++ -std=c++17 -Wall -Wextra -pedantic metro.cpp -o metro

Explanation:

- g++ invokes the GNU C++ compiler.
- -std=c++17 selects the C++17 standard.
- -Wall enables many useful warnings.
- -Wextra enables additional warnings.
- -pedantic requests diagnostics for non-standard extensions.
- metro.cpp is the source file.
- -o metro specifies the executable name.

Compiler warnings help identify suspicious code, unused variables, and potential mistakes. Testing is still necessary even when the program compiles without warnings.

10.3 Efficiency Analysis

The basic fare-calculation algorithm uses a fixed number of comparisons and arithmetic operations.

Its time complexity is:

$$□(□) = □(1)$$

Calculating the fare for one journey requires a fixed amount of work, so the time complexity is constant.

The basic calculation also uses a fixed number of variables:

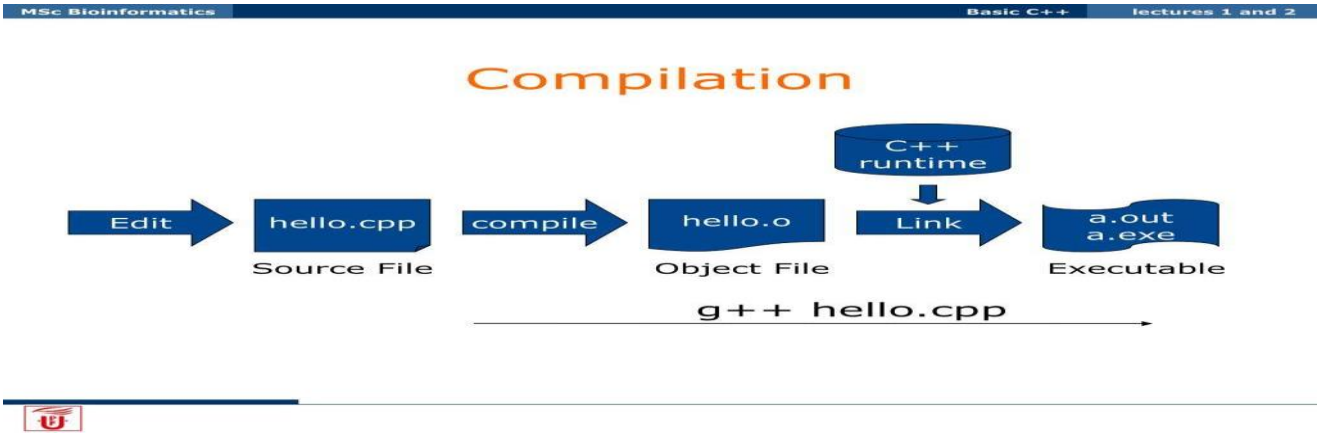
$$□(□) = □(1)$$

Therefore, the auxiliary space used for one fare calculation is constant. If the system stores journey history, the storage requirement increases with the number of recorded journeys. A history containing □ journeys may require □(□) storage.

10.4 Improving Code Quality

The following practices improve the quality of the implementation:

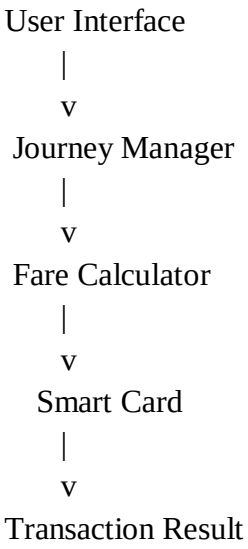
- Use meaningful variable and function names.
- Avoid repeating fare values throughout the program.
- Store fixed rules in constants.
- Separate fare calculation from payment processing.
- Validate user input before calculating the fare.
- Use functions that perform one clear task.
- Avoid unnecessary global variables.
- Test all boundary conditions.
- Document business rules that may change.
- Use a consistent money representation.



Task 11: Recommended Object-Oriented Architecture

11.1 Proposed Architecture

A maintainable system should separate its responsibilities into distinct components.



The user interface accepts the zone, time, and card selection. The Journey Manager validates journey details. The Fare Calculator applies zone fares and peak-hour rules. The Smart Card checks the balance and processes payment. Finally, the Transaction Result displays the fare, remaining balance, and status.

This is a logical design proposal; a graphical interface or database is not required for the basic C++ assignment.

11.2 Responsibilities of Each Class

SmartCard Class

The SmartCard class stores the current balance and manages payment operations.

Its responsibilities include:

- Storing the balance.
- Returning the balance through a controlled method.
- Checking whether a fare can be paid.

- Deducting the fare only when payment is valid.
- Preventing direct modification of the private balance.

Journey Class

The Journey class represents a particular journey. It can store the selected zone, travel time, calculated fare, and journey status.

A journey object keeps the details of one transaction together instead of scattering them across unrelated variables.

FareCalculator Class

The FareCalculator class calculates the fare based on the selected zone and travel time. It should not directly change the smart-card balance.

FareTable Class

The FareTable class stores the base fare for each zone. If the metro authority changes the fares, the fare table can be updated without changing the card-payment logic.

11.3 Advantages of Separation of Responsibilities

Maintainability: Changes to fare rules can be made in the fare-calculation component.

Reusability: The same smart-card class can be used for multiple passengers and journeys.

Testability: Each component can be tested separately. For example, fare calculations can be tested without creating a user interface.

Security of state: Encapsulation prevents external code from arbitrarily modifying the card balance.

Extensibility: Features such as monthly passes, refunds, travel history, and transaction receipts can be added later.

11.4 Possible Future Enhancements

The basic system can be extended with:

1. Card recharge to add money to a card.
2. Transaction history to store the date, zone, fare, and balance.
3. Concession passes for eligible passengers.

4. Monthly passes with defined travel rules.
5. Multiple smart cards and passenger identifiers.
6. Refund management.
7. Station-to-station fare calculation.
8. Database integration for card balances and transaction records.
9. Automated testing of fare rules.
10. Audit logs for important transaction events.

These are possible enhancements, not features already provided by the basic implementation.

Task 12: Extended Fare Rules and Object-Oriented Implementation

This task extends the original system to support an additional Airport Zone and a weekend discount. The following rules are illustrative assignment requirements, not official metro fare policies.

12.1 Additional Requirements

- Inner Zone fare: ₹10.
- Middle Zone fare: ₹20.
- Outer Zone fare: ₹30.
- Airport Zone fare: ₹60.
- Peak-hour surcharge: 20%.
- Weekend discount: 10% of the base fare, replacing the peak surcharge.
- Minimum remaining card balance: ₹50.

The Airport Zone has a base fare of ₹60. On weekdays during peak hours, a surcharge is applied. On weekends, the 10% discount is applied instead of the peak surcharge.

For this example, a weekend journey receives the discount regardless of whether the travel time is within a peak period.

12.2 Fare Calculation Examples

Example 1: Airport Zone on a weekday outside peak hours

Base fare:

$$₹60$$

No peak surcharge is applied.

$$\square\square\square\square\square\square\square\square\square\square = ₹60$$

Example 2: Airport Zone on a weekday during peak hours

The surcharge is 20% of ₹60.

$$\square\square\square\square h\square\square\square\square = 60 \times \frac{20}{100} = ₹12$$

Therefore:

$$\square\square\square\square\square\square\square\square\square\square = ₹60 + ₹12 = ₹72$$

Example 3: Airport Zone on a weekend

The discount is 10% of ₹60.

$$\square\square\square\square\square\square\square\square\square = 60 \times \frac{10}{100} = ₹6$$

Therefore:

$$\square\square\square\square\square\square\square\square\square\square = ₹60 - ₹6 = ₹54$$

12.3 Extended C++ Program

```
#include <iostream>
using namespace std;

constexpr int MIN_BALANCE = 5000;

int calculateFare(int zone, int hour, bool weekend) {
   int baseFare;

   switch (zone) {
      case 1:
         baseFare = 1000;
         break;
      case 2:
         baseFare = 2000;
         break;
      case 3:
         baseFare = 3000;
         break;
      case 4:
```

```
            baseFare = 6000;
            break;
        default:
            return -1;
    }

    if (hour < 0 || hour > 23)
        return -1;

    if (weekend)
        return baseFare - baseFare / 10;

    bool peak =
        (hour >= 8 && hour < 10) ||
        (hour >= 17 && hour < 20);

    if (peak)
        return baseFare + baseFare / 5;

    return baseFare;
}

class SmartCard {
private:
    int balancePaise;

public:
    SmartCard(int initialBalance)
        : balancePaise(initialBalance) {}

    int getBalance() const {
        return balancePaise;
    }

    bool payFare(int farePaise) {
        if (farePaise < 0 ||
            balancePaise - farePaise < MIN_BALANCE) {
            return false;
        }
```

```
        balancePaise -= farePaise;
        return true;
    }
};

int main() {
    SmartCard card(20000);

    int weekdayPeakFare = calculateFare(4, 18, false);
    int weekendFare = calculateFare(4, 12, true);

    cout << "Weekday peak Airport fare: "
         << weekdayPeakFare / 100 << "."
         << (weekdayPeakFare % 100 < 10 ? "0" : "")
         << weekdayPeakFare % 100 << '\n';

    cout << "Weekend Airport fare: "
         << weekendFare / 100 << "."
         << (weekendFare % 100 < 10 ? "0" : "")
         << weekendFare % 100 << '\n';

    if (card.payFare(weekdayPeakFare))
        cout << "Weekday journey approved\n";
    else
        cout << "Weekday journey rejected\n";

    if (card.payFare(weekendFare))
        cout << "Weekend journey approved\n";
    else
        cout << "Weekend journey rejected\n";

    cout << "Remaining balance: Rs "
         << card.getBalance() / 100 << "."
         << (card.getBalance() % 100 < 10 ? "0" : "")
         << card.getBalance() % 100 << '\n';

    return 0;
}
```

12.4 Expected Output

Weekday peak Airport fare: 72.00
Weekend Airport fare: 54.00
Weekday journey approved
Weekend journey approved
Remaining balance: Rs 74.00

The card starts with ₹200. The weekday peak journey costs ₹72, and the weekend journey costs ₹54.

$$₹200 - ₹72 - ₹54 = ₹74$$

The remaining balance is ₹74, which satisfies the minimum-balance rule of ₹50. Therefore, both journeys are approved.

12.5 Design Improvements for a Larger System

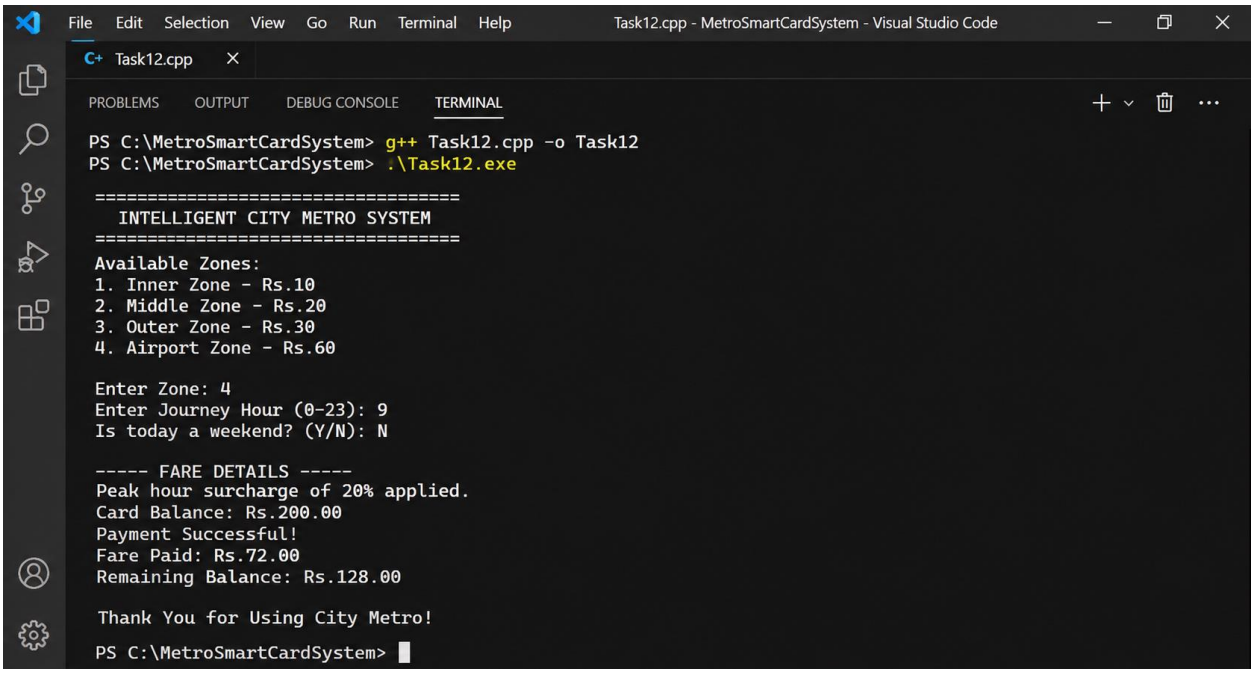
For a larger application, fare calculation should be separated into smaller functions or classes so that fare policies can be changed without modifying the card-payment mechanism.

Weekend discounts, peak-hour surcharges, and concession rules should be represented as explicit policies rather than a long collection of nested conditions.

A production-quality design should also specify:

- Whether discounts are calculated before or after surcharges.
- Whether discounts can be combined.
- How monetary rounding is performed.
- Whether the minimum balance is checked before or after all deductions.
- How invalid transactions are recorded.
- How fare-table changes are applied.

These details must be defined before implementing a real ticketing system.



Overall Conclusion

The City Metro Smart-Card System demonstrates the practical application of C++ programming concepts to a real-world problem. The project covers requirement analysis, algorithm development, pseudocode preparation, program implementation, validation, testing, and architectural design.

The assignment explains C++ tokens, constants, enumerations, data types, operators, expressions, and type conversion. It also compares procedural programming with object-oriented programming and demonstrates how classes can represent real-world entities such as smart cards and journeys.

Encapsulation helps protect the card balance. Input validation prevents invalid zone and time values from entering the fare-calculation process. Integer-based monetary representation helps avoid common floating-point precision problems, while boundary testing verifies that peak-hour and minimum-balance rules behave correctly.

The project also demonstrates how the design can be extended to support additional zones, concession passes, journey history, and other features. Separating fare calculation, journey management, and payment processing improves maintainability and makes future changes easier.

In conclusion, this assignment provides practical experience in applying fundamental C++ and OOP concepts to the design of a reliable, modular, and testable fare-calculation system. It establishes a foundation for developing more advanced ticketing applications with persistent data, multiple smart cards, automated testing, and additional fare policies.